

hubbleAI

Treasury cash-flow forecasting for Aperam. Weekly 8-week-ahead forecasts of receipts (TRR) and payments (TRP) per legal entity, with P10/P50/P90 uncertainty bands. For TRP horizons 1–4, the system also blends ML predictions with the existing Liquidity Plan into a hybrid forecast. A Streamlit UI in the same repo lets Treasury staff review the latest forecast, compare against LP, and trigger new runs.

This README is the starting point. Once it works, read docs/SYSTEM_GUIDE.md for the end-to-end walkthrough — sections 1–3 + 18 are the 5-minute version.

Document map

If you want to ...	Read
Install and run V1 locally (you are here)	this file
Understand the end-to-end pipeline	docs/SYSTEM_GUIDE.md (and the matching PDF)
Know what V1's known issues / landmines are	docs/SYSTEM_GUIDE.md §18
See the latest backtest performance	docs/SYSTEM_GUIDE.md §12.1
Re-deploy to Azure	docs/SYSTEM_GUIDE.md §16
Get the formal technical spec	Claude.md
See how the system was developed from scratch	notebooks/TCF_V2.ipynb

Requirements

- **Python 3.11** (the package targets 3.11 specifically; some dependencies pin Python ranges).
- Windows or Linux. macOS works but isn't actively tested.
- Roughly 3 GB of free disk for the virtual environment and the parquet outputs from a few backtest runs.

Install

```
# From the repo root, on Windows PowerShell:
python -m venv .venv
.\.venv\Scripts\Activate.ps1

pip install -r requirements.txt
pip install -e .
```

`pip install -e .` installs the `hubbleAI` package in editable mode so imports like `from hubbleAI.pipeline import run_forecast` work from anywhere.

On Linux / macOS / Git Bash:

```
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
```

Required data files

The pipeline reads exactly three CSV files from `data/raw/`. The filenames are pinned in `src/hubbleAI/config.py`. The shipped repo already includes a working set; you can skip this section if you just want to try the system.

File	Source	Required columns
<code>New_Actuals_17C7_2014.csv</code>	Treasury — historical actuals export	Entity, Value Date, Amount Functional Currency, Liquidity Group, Counterpart, Status, ISO Country Code
<code>New_LP_17C7.csv</code>	Treasury — Liquidity Plan export	Entity, Entity Name, Liquidity Group/Super Liquidity Group, Year Title, Item's Date, Amount, Currency, Plan Currency, Amount in plan currency, Rate, Comment
<code>20251120_eurofxref-hist.csv</code>	ECB FX reference rates	Date, USD, CHF

If you replace any of these, keep the filenames the same — or update `config.py:32-34`. The Admin page in the Streamlit app validates required columns on upload, so a wrongly-shaped file will be rejected.

Run the Streamlit UI

```
streamlit run app/streamlit_app.py
```

This serves the app on `http://localhost:8501`. Five pages:

- **Overview** — KPI tiles + nav cards.
- **Cash Flows** — the latest 8-week forecast with P10/P50/P90 bands, split into TRR / TRP / NET tabs.
- **Performance Dashboard** — weekly WAPE for ML vs LP, with target lines and a quantile-calibration table.
- **Backtest Explorer** — pick a historical week and inspect the H1–H8 forecast made from it.
- **Admin** — the only write-capable page. "Run Forward Forecast", "Run Backtest", and CSV upload widgets for the three input files.

Run a forecast from Python

```
from hubbleAI.pipeline import run_forecast

# Forward mode: produces an 8-week forecast for the latest Monday in the data.
status = run_forecast(mode="forward", trigger_source="manual")
```

```
print(status)

# Backtest mode: 85/10/5 split, evaluates on the last 5% of weeks, produces metrics + diagnostics.
status = run_forecast(mode="backtest", trigger_source="manual")
print(status)
```

Outputs land under `data/processed/`:

- Forward predictions → `data/processed/forecasts/{ref_week_start}/forecasts.parquet`
- Backtest predictions → `data/processed/backtests/{ref_week_start}/backtest_predictions.parquet`
- Backtest metrics → `data/processed/metrics/backtests/{ref_week_start}/*.parquet`
- Run-status JSON → `data/processed/run_status/...`

A typical forward run trains 64 LightGBM models (point + 3 quantile models × 2 liquidity groups × 8 horizons) and takes 2–5 minutes on a typical laptop.

Run the tests

```
python -m pytest tests/ -v
```

The test suite covers the `evaluation/metrics.py` module — WAPE primitives and the hybrid- α tuning workflow (23 tests). The rest of the code is not yet covered by automated tests; see `docs/SYSTEM_GUIDE.md §18.4`.

Troubleshooting

"ModuleNotFoundError: No module named 'hubbleAI'" You probably skipped `pip install -e ..`. Re-run it from the repo root with the virtual environment activated.

"Streamlit version too old" `pip install -r requirements.txt` should pull a modern Streamlit (≥ 1.28). If you started from a pre-existing environment, force-upgrade with `pip install -U streamlit`.

"Data Status: Incomplete" on the Overview page One or more of the three CSVs is missing from `data/raw/`. Check filenames match `config.py:32-34` exactly. Re-upload through the Admin page if needed.

Forecasts run takes 5+ minutes Expected. The pipeline retrains 64 LightGBM models on every run; there's no model persistence in V1. The receipt of a "Forecast completed!" toast in the Admin page is the success signal.

Project layout (short version)

hubbleAI/	
├ README.md, Claude.md	docs
├ docs/SYSTEM_GUIDE.md, .pdf	
├ data/raw/	input CSVs
├ data/processed/	all outputs (parquet + JSON)
├ src/hubbleAI/	the package
│ └ pipeline.py	<code>run_forecast()</code> – single entry point
│ └ data_prep/	load, FX, aggregate, merge
│ └ features/	lag, rolling, calendar, trend, LP
│ └ models/lightgbm_model.py	training + prediction
└ evaluation/metrics.py	WAPE, MAE, pinball, hybrid α

		service.py	read-only helpers for the UI
		config.py	constants + Tier-2 list
		app/	Streamlit UI
		notebooks/TCF_V2.ipynb	original development notebook
		tests/test_metrics.py	unit tests

For everything else, [docs/SYSTEM_GUIDE.md](#) is the right next stop.